Ukrainian Catholic University

Bachelor Thesis

CXU Playground: RISC-V Custom Extension Development Framework

Author:
Petro Mozil

Supervisor:
Dr. Oleh Farenyuk

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences

Lviv 2026

"*Thanks to my solid academic training, today I can write hundreds of words on virtually any topic without possessing a shred of information, which is how I got a good job in journalism.*"

Dave Barry

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

CXU Playground: RISC-V Custom Extension Development Framework

by Petro Mozil

Abstract

The Thesis Abstract is written here (and usually kept to just this page). The page is kept centered vertically so can expand into the blank space above the title too...

Acknowledgements

The acknowledgments and the people to thank go here, don't forget to include your project advisor...

Contents

Abstract **ii**

Acknowledgements **iii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Contributions . . . 1
1.3 Structure Of The Thesis . . . 2

2 Related Work **3**
2.1 RISC-V Extensions For Custom Instruction Integration . . . 3
2.1.1 CFU . . . 3
2.1.2 CXU . . . 3
2.1.3 Core-V-XIF . . . 4
2.2 Frameworks for FPGA-based SOC development . . . 4
2.2.1 LiteX . . . 4
2.2.2 F4PGA . . . 4
2.2.3 MiSoC . . . 4
2.2.4 PULP Platform . . . 5
2.2.5 SpinalHDL . . . 5
2.3 Sandboxes for simplified development of custom instruction hardware . 5
2.3.1 FRANCIS-V . . . 5
2.3.2 CFU Playground . . . 5
2.4 Applications of Custom Instruction Extensions . . . 6
2.4.1 AES and Symmetric Cryptography . . . 6
2.4.2 Post-Quantum Cryptography . . . 6
2.4.3 Hashing . . . 7
2.4.4 Data Compression . . . 7
2.4.5 Machine Learning Inference . . . 7
2.4.6 Linux on LiteX . . . 7

3 Theoretical Background **9**
3.1 The RISC-V Architecture and VexiiRiscv Core . . . 9
3.1.1 Overview of the RISC-V ISA . . . 9
3.1.2 The VexiiRiscv Implementation . . . 9
3.2 The LiteX SoC Ecosystem . . . 10
3.2.1 LiteX Framework Overview . . . 10
3.2.2 Memory-Mapped Architecture . . . 10
3.2.3 Interconnect and Control . . . 10
3.2.4 Memory Management . . . 11
3.3 Software Development for LiteX-Based SoCs . . . 11
3.3.1 Cross-Compilation . . . 11
3.3.2 Linker Scripts . . . 11

3.3.3 Bare-Metal vs Linux-Based Systems . . . 11
3.4 Buildroot . . . 11
3.5 Custom Extension Units (CXU) Mechanism . . . 12
3.5.1 Coupling Architecture . . . 12
3.5.2 VexiiRiscv CXU User Plugin Interface . . . 12
Instruction Decoding . . . 12
3.6 Linux Support on LiteX . . . 13
3.6.1 Hardware Requirements . . . 13
3.6.2 The Boot Process . . . 13
3.6.3 Device Tree Overlay (DTS) . . . 13
3.7 Lattice-Based Cryptography . . . 13
3.7.1 Post-Quantum Context . . . 13
3.8 The AES Encryption . . . 14
3.8.1 AES Round Operations . . . 14
KeyExpansion . . . 14
AddRoundKey . . . 15
MixColumns . . . 15
3.8.2 Speeding up AES with the CXU . . . 15
3.9 GZIP . . . 15
3.9.1 The DEFLATE Algorithm . . . 15
3.9.2 LZ77 Compression . . . 15
3.9.3 Huffman Coding . . . 16
3.9.4 Dynamic vs Static Huffman Trees . . . 16
3.9.5 Custom Instructions for DEFLATE Acceleration . . . 16
Bit Reversal . . . 16
Bit Mask Generation . . . 17
Huffman Table Index Extraction . . . 17
LZ77 Back-Reference Address Calculation . . . 17
Implementation Characteristics . . . 17
3.10 TensorFlow Lite Micro . . . 18
3.10.1 Multiply-Accumulate Operations . . . 18

4 Proposed Solution **21**
4.1 Problem Statement . . . 21
4.2 System Architecture and Playground Design . . . 21
4.3 CXU Integration in VexiiRiscv . . . 22
4.4 Sandbox Development Environment and Buildroot Integration . . . 23
4.5 The CXU Runtime . . . 23
4.6 Validation through Case Studies . . . 23
4.7 Testing and Benchmarking . . . 24
4.8 Testing and Benchmarking Methodology . . . 24
4.9 Data Analysis . . . 25
4.10 Limitations . . . 25

5 Experiments and Results **27**
5.1 The CXU Playground . . . 27
5.1.1 The CXU interface modification . . . 27
5.1.2 The LiteX SoC development environment . . . 28
5.1.3 The environment for bare-metal application development . . . 28
5.1.4 The AES implementation . . . 28
5.1.5 The GZIP implementation . . . 28

5.1.6 TensorFlow Lite Micro MAC speedup 28
5.2 Experiments and Benchmarks 28
5.2.1 AES $GF(2^8)$ extension and software comparison 28
5.2.2 GZIP extension and software implementation 29
5.2.3 TensorFlow MAC extension 29
5.3 Unimplemented Features and Known Limitations 29
5.3.1 CXU Capabilities in the Linux Device Tree 29
5.3.2 Clock Frequency Limitations 30
5.3.3 Multi-Process CXU Access and Kernel Driver 30

6 Conclusions **32**
6.1 Summary of Contributions 32
6.2 Key Results 32
6.3 Limitations of the Current Implementation 33
6.4 Future Work 33
6.5 Closing Remarks 34

Bibliography **35**

List of Figures

2.1 FRANCIS-V framework overview. 5
2.2 CFU Playground full-stack architecture. 6

3.1 Hardware architecture of the generated LiteX SoC 10
3.2 CXU-CPU connection 12
3.3 VexiiRiscv CXU operation diagram. 13
3.4 the LiteX soc linux boot process 14
3.5 Flowchart of the AES encryption algorithm 19
3.6 Flowchart of the DEFLATE compression pipeline 20

4.1 Integration of the CXU as a tightly coupled plugin within the VexiiRiscv pipeline 22
4.2 The CXU Sandbox full-system stack 23

5.1 Runtime comparison of GZIP runtime for different file sizes 31

List of Tables

5.1 Runtime comparison of AES runtime for different buffer sizes 29
5.2 Throughput comparison of AES runtime for different buffer sizes . . . 30
5.3 Inference latency comparison for TFLite Micro using the CXU MAC extension. 30

List of Abbreviations

ABI	**A**pplication **B**inary **I**nterface
AES	**A**dvanced **E**ncryption **S**tandard
CFU	**C**omposable **F**unction **U**nit
CPU	**C**entral **P**rocessing **U**nit
CSR	**C**ontrol/**S**tatus **R**egister
CXU	**C**omposable **X**tension **U**nit
DTB	**D**evice **T**ree **B**inary
DTS	**D**evice **T**ree **S**ource
MMU	**M**emory **M**anagement **U**nit
PQC	**P**ost **Q**uantum **C**ryptography
RVTSO	**R**ISC-**V** **T**otal **S**tore **O**rdering
RVWMO	**R**ISC-**V** **W**eak **M**emory **O**rdering
SBI	**S**upervisor **B**inary **I**nterface
SoC	**S**ystem **o**n **C**hip
XLEN	The length of a RISC-V general-purpose register in bits fo a given CPU

For/Dedicated to/To my...

Chapter 1

Introduction

1.1 Motivation

RISC-V [31] is a modern and extensible instruction set architecture often used in systems-on-chip, especially, FPGA-based ones [9], such that it is both extensible and energy-efficient. RISC-V's instruction set architecture is designed with custom extensions in mind, so designing custom extensions would be a straightforward process. It is, however, a difficult task to integrate a custom extension into the RISC-V specification, and even more so to implement an extension for the multitude of RISC-V processor implementations. With such issues, and the fact that most FPGA-based SOCs offer plenty of space to also implement an extension, in mind, Jan Gray et al. designed the custom composable extensions (CFU) [16] extension. It allows for an implementation of a specific extension to be used across multiple RISC-V CPUs, so as to use the single extension implementation across a multitude of different RISC-V CPUs. CFU allows for a single extension to be connected to a hardware thread. It also allows passing a "function index" to an extension, such that up to 255 different functions may be implemented.

There is also a playground for desgining the CFU extensions on a ready-made SOC called the CFU playground. It allows the users to implement a CFU and write a main script that runs and invokes the implemented extension. CFU is a useful interface for implementing a single extension, but when combining extensions, especially those, that may have multiple different functions and utilize the function id, it becomes incredibly difficult and cumbersome to switch between them. Because of that, Gray et al. also designed a newer version of composable custom extensions - the CXU [17]. This implementation allows for switching between multiple custom extensions, but, as of now, there is not yet an implementation of it on any of the RISC-V implementations, nor an environment for developing custom extensions.

1.2 Contributions

This thesis makes the following concrete contributions:

- **First CXU implementation:** A SpinalHDL plugin for the VexiiRiscv processor that realises the CXU interface, including opcode interception at the execute stage, pipeline stall management for multi-cycle operations, and `cxidx`/`cxdata` CSR support.
- **CXU Playground:** A unified, open-source sandbox built on LiteX that automates SoC generation, Buildroot Linux integration, cross-compilation, and CXU hardware templating, enabling developers to prototype and benchmark custom extensions under realistic, OS-level conditions.

- **C-level CXU runtime library:** A header-only library that wraps the low-level assembly invocation, manages extension multiplexing via the `cxidx` register, and provides a portable C API for CXU-accelerated routines.
- **Three case-study extensions:** An AES GF(2^8) accelerator, a GZIP/DEFLATE bit-manipulation accelerator, and a multiply-accumulate (MAC) extension for TensorFlow Lite Micro inference, each accompanied by a paired software reference implementation and benchmarking suite.

Benchmarking results demonstrate approximately a 5.25× reduction in cycles per byte for AES encryption and a roughly 10% reduction in CPU time for GZIP bitstream manipulation, confirming that even a small number of tightly coupled custom instructions can yield substantial gains for arithmetic-intensive workloads.

1.3 Structure Of The Thesis

- **Chapter 2** surveys existing custom-instruction interfaces (CFU, CXU, Core-V-XIF), FPGA-based SoC frameworks, related sandboxes, and prior work applying custom ISA extensions to cryptography, compression, and machine learning.
- **Chapter 3** provides the theoretical background covering the RISC-V ISA, the VexiiRiscv pipeline, the LiteX SoC ecosystem, the CXU coupling mechanism, Linux boot flow on LiteX, and the algorithmic foundations of the three case studies.
- **Chapter 4** describes the design of the CXU Playground: the system architecture, the CXU plugin integration in VexiiRiscv, the Buildroot environment, the runtime library, and the benchmarking methodology.
- **Chapter 5** presents the implementation details and experimental results for the AES, GZIP, and TFLite Micro extensions, and discusses unimplemented features and known limitations.
- **Chapter 6** summarises the contributions, reflects on the limitations of the current implementation, and outlines directions for future work.

Chapter 2

Related Work

In this chapter, we review frameworks for development of FPGA-based SOCs, works that explore hardware-based optimizations for certain algorithms and works that explore other ways of managing multiple extensions for RISC-V-based CPUs.

2.1 RISC-V Extensions For Custom Instruction Integration

In this section, we provide an overview of what the composable custom extensions specification is and what it may be used for.

2.1.1 CFU

Composable Custom Extensions [16, 17, 18] is a RISC-V extension proposed by Jan Gray et al. It's an extension aimed at making custom extensions modular and usable between different implementations of RISC-V CPUs. The motivation behind the composable custom extensions is that many RISC-V CPUs use some sort of custom extensions, yet there is no cross-implementation interoperability, and so there is many implementations of a single extension (e. g. SIMD multiply-accumulate) across multiple RISC-V CPU implementations. One solution to that is to provide a common logic interface to connect extensions to, and that is exactly the Composable Custom Extensions RISC-V ISA extension.

The first version of composable custom extensions if the Custom Function Unit (CFU). It was proposed in 2020 by Jan Gray et al. and provides an interface to connect custom extensions to along with timing, logic and combinatorial constraints for what an extension may do [16]. We will go over what constitutes a composable extension in the theoretical background chapter.

The CFU allows for a single extension to be used with a CPU, and it requires the user to invoke a **custom-0|1|2|3** instruction, even when not using assembly, such that when using a higher-level language, the user would still need to invoke an assembly instruction. The CFU also may not access memory. [16]

2.1.2 CXU

The CXU extension is a newer version of composable custom extensions. It allows for up to $2^{\mathbf{XLEN}}$ custom extensions. It also allows for extensions to have a shared state and defines the limitations under which an extension may have access to memory. [17]

Notably, the CXU specification also defines the ABI, making it possible to call the extension from a higher-level language without invoking the assembly instruction directly. An example of a CXU runtime is also provided, such that calls to the CXU extensions would be wrapped in helper functions that select a proper extension,

save the previous state and clean up the context afterwards. The specification was developed with support for operating systems in mind, and it specifies how the OS and process context switching is to be done with respect to the CXU state as well.

2.1.3 Core-V-XIF

Core-V E**X**tensible **I**nter**F**ace [24] is a RISC-V extension interface for implementing custom ISA extensions and coprocessors. As composable custom extensions, it's an interface that allows for custom extensions to be integrated into any CPU that supports the Core-V-XIF interface.

Unlike composable custom extensions, Core-V-XIF allows for custom CSRs and overall is less restrictive. Like the CFU Playgroud [28], there is an implementation of a sandbox for custom extensions, Francis-V [5]. However, it's more tightly coupled to a certain CPU implementation, and would not allow for straightforward interoperability of implemented extensions.

2.2 Frameworks for FPGA-based SOC development

In this section, we provide an overview for frameworks that provide a base for developing FPGA-based SOCs. These frameworks usually provide a way to extend the SOC with additional functionality, be it an ISA extension or an IO device, e. g. a specialized USB port for debugging. The frameworks usually consist of modules, which are then assembled into a bigger system.

2.2.1 LiteX

LiteX [9] is an open-source python-based framework for building FPGA-based SoCs. It provides modules for the bus, IO and CPU integration (notably VexiiRiscv [27]). It includes support for most popular dram modules used on FPGA devboards, support for ethernet, PCIE and SATA buses, support for SD card readers, inter-chip communication and most popular serial communication buses, notably I2C, UART, SPI and USB.

The most popular implementation of an environment for development of CFUs, the CFU Playground [28], used LiteX as a basis for it's environment.

We use litex for our implementation as well.

2.2.2 F4PGA

F4PGA [11] provides a fully open-source FPGA toolchain targeting multiple FPGA families. While not a SoC framework by itself, it is commonly used together with LiteX or custom SoC designs to enable reproducible, vendor-independent FPGA-based SoC development.

2.2.3 MiSoC

MiSoC [2] is the predecessor of LiteX and introduced Python-based SoC generation concepts. Although deprecated, it is historically important and illustrates early approaches to modular FPGA SoC construction, which directly influenced LiteX's architecture.

2.2.4 PULP Platform

PULP [32] is an open-source RISC-V-based SoC platform focused on energy-efficient parallel processing and accelerator integration. It uses a cluster-based architecture and is widely used in academic research for DSP, cryptography, and accelerator-centric SoC exploration.

2.2.5 SpinalHDL

SpinalHDL [26] is a hardware construction language often used to build extensible SoCs, most notably the VexRiscv core. Its plugin-based CPU architecture enables fine-grained custom instruction and pipeline extensions, making it highly relevant for composable extension research.

SpinalHDL is used in the **VexiiRISCV** [27] implementation, and we use VexiiRISCV as our CPU.

2.3 Sandboxes for simplified development of custom instruction hardware

2.3.1 FRANCIS-V

FRANCIS-V[5] (FRAmework for iNtegrating Custom InstructionS into RISC-V systems) [5] is a framework aimed as making development and integration of custom extensions and coprocessors easier while designing RISC-V processors. It was proposed to make enginnering custom instructions easier.

The framework provices an interface and methodology for connecting custom instruction hardware to a processor using the **Core-V-XiF** extension. It supports simulation, compilation and verificcation workflows. It allows users to automate most parts of development, much like the **CFU playground** and allows for customized RISC-V processors.

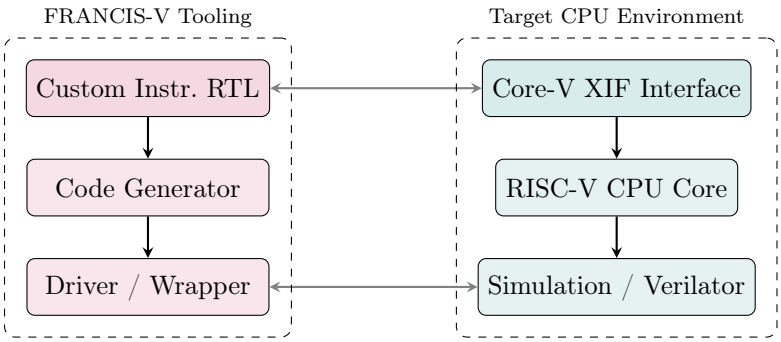

FIGURE 2.1: FRANCIS-V framework overview.

2.3.2 CFU Playground

The CFU Playground [28] is an environment for development of custom compute units on a litex SoC.

The CFU Playground [28] is a full-stack open-source playground designed for the rapid design and evaluation of Custom Function Units (CFUs) to accelerate FPGA-based RISC-V soft CPUs.

Originally proposed for Tiny Machine Learning (tinyML) acceleration, it utilizes the LiteX framework as its architectural base to provide a ready-made SoC environment [9]. The playground allows users to implement specialized hardware logic and write software scripts to invoke these extensions.

The architecture is based on the initial version of the Composable Custom Extension specification, which connects a single extension to a hardware thread [16, 28] and, usually a single CPU. While the interface supports up to 255 different function indices, it is characterized by some constraints: the unit may not access memory directly, and users must invoke instructions via assembly-level custom-0|1|2|3 opcodes, even when programming in higher-level languages [16]. These restrictions eventually led to the development of the more flexible CXU specification [17, 18].

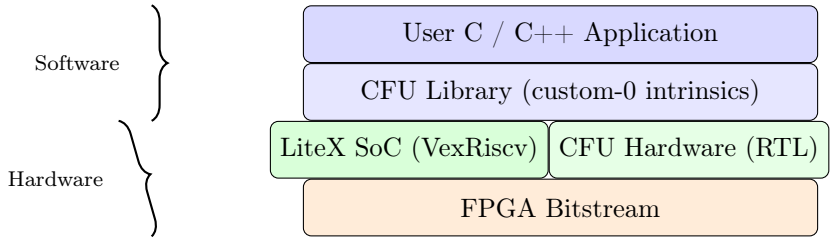

FIGURE 2.2: CFU Playground full-stack architecture.

2.4 Applications of Custom Instruction Extensions

The interface standards and sandboxes above exist to serve a practical need: a wide range of important algorithms benefit substantially from narrowly scoped hardware acceleration. This section surveys the application domains most relevant to the case studies in this thesis — cryptography, data compression, and machine learning — and draws out the lessons that inform the design of the extensions evaluated in Chapter 5.

2.4.1 AES and Symmetric Cryptography

AES [**NIST_AES**] is based on a substitution-permutation network whose inner operations — SubBytes (an S-box lookup over GF(2^8)) and MixColumns (matrix multiplication over GF(2^8)) — map naturally onto combinational hardware. Cui and Balasch [3] demonstrate this concretely in the context of masked AES, introducing constant-time finite-field instructions that simultaneously improve performance and resistance to side-channel attacks. Their result is especially relevant here because it shows that a small number of carefully scoped ISA extensions — rather than a monolithic AES coprocessor — can yield substantial gains while preserving software flexibility.

Primas et al. [29] take a related approach with the NLU-V extension, fusing S-box substitution and linear diffusion into a single hardware block to minimise intermediate data movement for both AES and PRESENT. Both works support the design choice made in Chapter 4 to expose GF(2^8) arithmetic as a primitive CXU instruction rather than implementing a full AES round.

2.4.2 Post-Quantum Cryptography

Classical public-key cryptosystems are vulnerable to Shor's algorithm on sufficiently powerful quantum computers, motivating the NIST post-quantum standardisation effort [35, 20]. Lattice-based schemes such as CRYSTALS-Kyber and NewHope rely on

polynomial arithmetic, particularly the Number Theoretic Transform (NTT), whose butterfly operations and modular reductions are expensive on a standard ALU.

Sozbilir et al. [33] prototype NTT-specific ISA extensions on VexRiscv and report up to an 85% speedup for Kyber and NewHope, demonstrating that the class of processor used in this work is a viable target for PQC acceleration. Levi et al. [20] extend this further with RISQ-V, a set of 29 tightly coupled custom instructions achieving a 9.6× speedup for Kyber. Franchetti et al. [15] address the related problem of multi-word carry-propagation arithmetic required by classical public-key schemes, using a coprocessor interface structurally similar to the CXU. Vranken et al. [35] argue for a uniform set of multiply-accumulate and modular-reduction primitives capable of serving both classical and post-quantum schemes, which foreshadows the approach taken for the MAC extension in Chapter 5.

These works collectively motivate extending the CXU Playground to post-quantum workloads as a natural direction for future work.

2.4.3 Hashing

Alawadi et al. [1] implement the full Keccak-f permutation as a single custom RISC-V instruction, reporting substantial reductions in instruction count and cycle time for SHA-3. Their area-performance trade-off analysis provides a useful reference for understanding the cost of integrating high-complexity primitives into the CXU.

2.4.4 Data Compression

GZIP/DEFLATE combines LZ77 back-reference encoding with Huffman entropy coding [**RFC1951**]. The performance bottleneck lies in the inner decompression loop, which requires repeated bit-field extractions, variable-width mask generation, and modular address arithmetic — all operations that reduce to a handful of hardware gates but require several ALU instructions in software. The custom instructions designed in Section 3.9 and evaluated in Chapter 5 target exactly this hot path. To our knowledge, no prior work has applied the CXU or CFU mechanism to GZIP acceleration, making this a novel case study.

2.4.5 Machine Learning Inference

Neural network inference on microcontrollers is dominated by Multiply-Accumulate (MAC) operations in convolutional and fully connected layers. TensorFlow Lite Micro [4] provides portable INT8-quantised inference kernels for this class of device. The CMSIS-NN library [19] shows that ARM DSP SIMD MAC instructions yield up to a 4.6× improvement over scalar code; the PULP platform [32] achieves comparable gains on RISC-V with hardware-loop and dot-product extensions. The MAC extension evaluated in Chapter 5 follows the same principle, exposing a fused multiply-accumulate operation as a single-cycle CXU instruction.

MiniFloat-NN and ExSdotp [23] push this further with float8 dot-product accumulation for low-precision training, demonstrating that the same CXU-style integration pattern scales to more exotic arithmetic formats.

2.4.6 Linux on LiteX

Running benchmarks inside a full operating system is important for obtaining performance numbers that reflect real-world deployment conditions rather than isolated

microbenchmarks. The Linux-on-LiteX-VexRiscv project [10] establishes the feasibility of booting Linux on a soft RISC-V core in a LiteX SoC, and documents the required hardware configuration (MMU, atomic instructions, supervisor privilege mode). This work adopts the same configuration and extends it with CXU hardware.

Chapter 3

Theoretical Background

In this chapter, we will look at the RISC-V ISA, analyze the main elements of developing a CPU with SpinalHDL [26] on the case of **VexiiRiscV**, look at the LiteX ecosystem and framework, how custom extensions are ususally implemented, and how software is run on the SoCs produced with such methods.

3.1 The RISC-V Architecture and VexiiRiscv Core

3.1.1 Overview of the RISC-V ISA

The RISC-V instruction set architecture (ISA) [31] is an open standard designed with simplicity, extensibility, and long-term sustainability in mind. Unlike proprietary architectures such as x86 or ARM, RISC-V is developed under an open governance model and can be freely implemented without licensing constraints.

A central design principle of RISC-V is modularity. The base ISA provides only essential functionality, while optional extensions add specialized capabilities. Common extension subsets include integer multiplication (**IM**), atomic operations (**A**), single and double precision floating point arithmetic (**F**/**D**) and compressed 16-bit instructions (**C**).

Typical Linux-capable systems use configurations such as RV64IMAC [30], which includes general-purpose, compressed, floating-point, and atomic extensions, however, for our implementation, we use RV32IMA. [10]

Importantly, RISC-V reserves opcode space for custom extensions. These custom instruction fields enable designers to introduce application-specific accelerators directly into the processor pipeline without conflicting with standard instructions. This extensibility forms the basis for the custom CXU interface described later in this chapter.

3.1.2 The VexiiRiscv Implementation

VexiiRiscv is a configurable RISC-V core implemented in SpinalHDL, a Scala-based hardware description language. Unlike traditional Verilog or VHDL, SpinalHDL supports generative and highly parameterized hardware construction. This approach allows for scalable design-space exploration, easier feature composition, and cleaner abstraction boundaries. [26, 27]

The processor follows a classical pipelined architecture. The VexiiRiscv processor supports up to 5 stages, **Fetch**, **Decode**, **Execute**, **Memory** and **Write-back**, with **Fetch** + **Execute** (possibly unified) and **Memory and Write-back** (also unified).

One of the distinguishing characteristics of VexiiRiscv is its plugin-based architecture. Major features—including memory management units (MMU), caches, branch predictors, and custom execution units—are implemented as independent plugins. The

CXU extension is an execute extension. All computation done by CXU coprocessors is done in the execution step of **VexiiRiscv**.

3.2 The LiteX SoC Ecosystem

3.2.1 LiteX Framework Overview

LiteX [9] is a python-based SoC construction framework built on top of Migen [2]. It enables automated generation of FPGA-based SoCs by integrating CPU cores, memory controllers, interconnects, and peripherals into a cohesive hardware platform.

Within this ecosystem, LiteX acts as the integration layer between the VexiiRiscv CPU core and FPGA peripherals such as UART, Ethernet [7], timers [9], and external memory controllers [6, 8]. The framework abstracts low-level hardware connectivity and provides a unified build system for generating bitstreams and associated firmware.

The framework automatically generates HDL, memory maps, CSR definitions and software headers.

3.2.2 Memory-Mapped Architecture

LiteX uses memory-mapped I/O. Peripherals are accessed by reading or writing specific addresses:

```
#define UART_BASE 0xF0001000
```

Software interacts with hardware by manipulating these addresses.

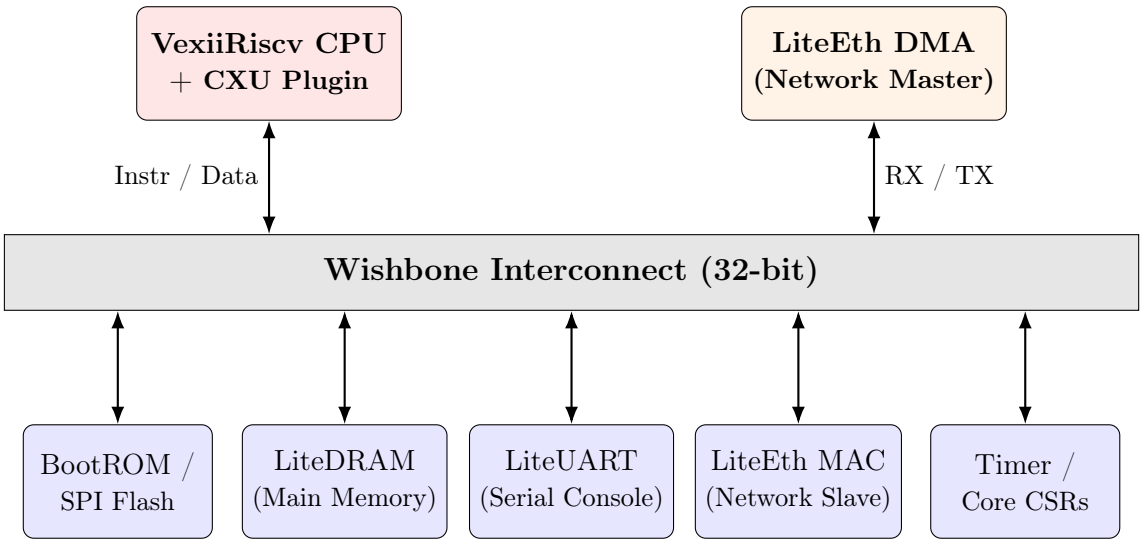

FIGURE 3.1: Hardware architecture of the generated LiteX SoC

3.2.3 Interconnect and Control

LiteX commonly uses the Wishbone bus as its on-chip interconnect standard. Wishbone is a simple and flexible synchronous bus protocol that supports master-slave communication between cores and peripherals. It enables memory-mapped I/O and simplifies peripheral integration. [9]

Control and Status Registers (CSRs) provide a structured mechanism for software-hardware communication. CSRs are memory-mapped registers that allow software to

configure hardware blocks, query status information, and trigger operations [6, 9]. This abstraction is essential for interacting with accelerators and managing runtime behavior.

3.2.4 Memory Management

LiteX integrates external memory controllers through LiteDRAM [6], which provides support for SDRAM and DDR memory devices. LiteDRAM abstracts memory timing, initialization, and refresh management, exposing a consistent interface to the system bus.

The SoC memory map defines address regions for ROM, SRAM, DRAM, CSRs, and peripherals. Proper memory layout is particularly important for Linux-based systems, where kernel placement, device tree configuration, and bootloader operation depend on predictable address organization.

3.3 Software Development for LiteX-Based SoCs

Developing software for a **LiteX**-based SoC requires cross-compilation and careful linking against the target memory map. Since LiteX systems typically use RISC-V softcores (e.g., VexRiscv), programs must be compiled for the appropriate architecture. [9]

3.3.1 Cross-Compilation

Software is compiled on a host machine using a toolchain for the desired risc-v architecrure.

The resulting ELF binary is then loaded into on-chip memory (SRAM, DRAM, or SPI Flash), as in [21].

3.3.2 Linker Scripts

LiteX SoCs define a memory map that must be respected during linking. A linker script specifies instruction memory region, data memory region and stack and heap placement.

Correct linking ensures the program executes from valid memory and interacts properly with memory-mapped peripherals.

3.3.3 Bare-Metal vs Linux-Based Systems

The LiteX platform supports bare-metal, RTOS-based and linux operating systems. Bare-metal software directly accesses memory-mapped registers, while Linux- and RTOS-based systems require drivers and standard system calls.

3.4 Buildroot

Buildroot is an embedded Linux build system used to generate complete root filesystems, toolchains, and kernel images for embedded targets.

Buildroot automates cross-toolchain generation, linux kernel config, rootfs and package selection. The workflow is as such: configuring the target, selecting the architecture, enabling desired packages and building the system complete image, while the output typically includes the kernel image, DTB and the filesystem image.

Buildroot is commonly used with LiteX SoCs to produce bootable Linux images for FPGA-based systems. It's used with [21].

3.5 Custom Extension Units (CXU) Mechanism

3.5.1 Coupling Architecture

Hardware accelerators can be integrated into a system using either tightly coupled or loosely coupled approaches. [16, 17]

Tightly coupled accelerators are directly integrated into the CPU pipeline. They operate as execution units and are invoked through custom instructions. This approach offers low latency and direct register-file access but requires careful hazard and pipeline management.

Loosely coupled accelerators are attached via system buses or DMA engines. They operate as independent hardware blocks, often accessed through memory-mapped interfaces. While simpler to integrate, they typically incur higher invocation latency.

The CXU mechanism adopts a tightly coupled strategy, enabling instruction-level acceleration with minimal software overhead.

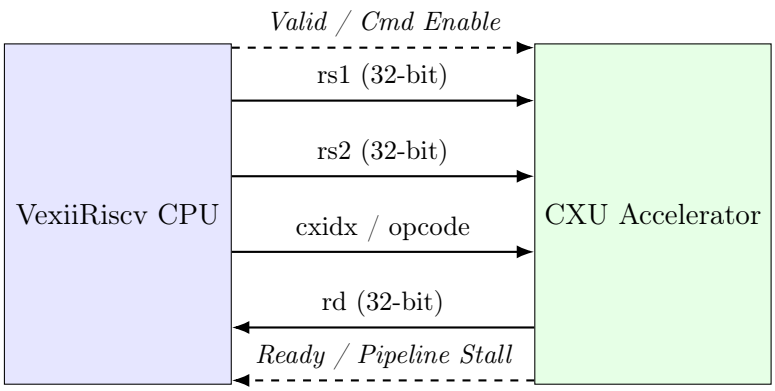

FIGURE 3.2: CXU-CPU connection

3.5.2 VexiiRiscv CXU User Plugin Interface

The VexiiRiscv plugin interface enables custom instruction integration at multiple levels of the pipeline.

Instruction Decoding

Specific opcode patterns are reserved and mapped to the CXU [26]. During the decode stage, matching instructions are redirected to the custom execution logic by SpinalHDL's built-in extension logic, after which the CXU connector extension sends the decoded command to the CXU coprocessor.

Multi-cycle CXU operations require pipeline stall mechanisms to preserve correctness. The plugin must signal readiness and manage structural, data, and control hazards to ensure correct execution ordering.

1. CXU state is updated through the use of *cxidx* and *cxdata* CSRs.
2. The CXU custom command is called with $rs1$ and $rs2$ as inputs and rd as output.
3. Source operands are read from registers $rs1$ and $rs2$.
4. The CXU processes the input state and registers and returns the desired output.
5. The result is written back to the destination register rd.
6. Updated CXU state may be read from through the use of *cxidx* and *cxdata* CSRs.

FIGURE 3.3: VexiiRiscv CXU operation diagram.

3.6 Linux Support on LiteX

3.6.1 Hardware Requirements

Running Linux on a RISC-V SoC requires [10] A Memory Management Unit (MMU) for virtual memory, atomic instruction support (RV32A/RV64A) for synchronization primitives and a supervisor privilege mode.

These features are enabled through appropriate VexiiRiscv plugins and LiteX SoC configuration parameters.

3.6.2 The Boot Process

The Linux boot flow typically involves multiple stages. OpenSBI [13] provides the Supervisor Binary Interface (SBI), acting as a firmware layer between machine mode and the operating system. It handles low-level initialization and exposes standardized services to the kernel.

A secondary bootloader such as U-Boot [25] loads the Linux kernel image and optional initial ramdisk into memory. Once control is transferred, the Linux kernel initializes drivers, memory management, and user-space services.

3.6.3 Device Tree Overlay (DTS)

The device tree describes the hardware layout to the Linux kernel [14]. LiteX generates a hardware description that is translated into a Device Tree Source (DTS) file. This file specifies CPU configuration, memory regions, peripheral addresses, and interrupt mappings. The kernel uses this information to configure drivers and manage hardware resources dynamically.

3.7 Lattice-Based Cryptography

3.7.1 Post-Quantum Context

Classical public-key cryptosystems such as RSA and elliptic curve cryptography rely on number-theoretic problems vulnerable to Shor's algorithm on sufficiently powerful quantum computers. This motivates the development of post-quantum cryptographic schemes. [35, 20, 33, 15]

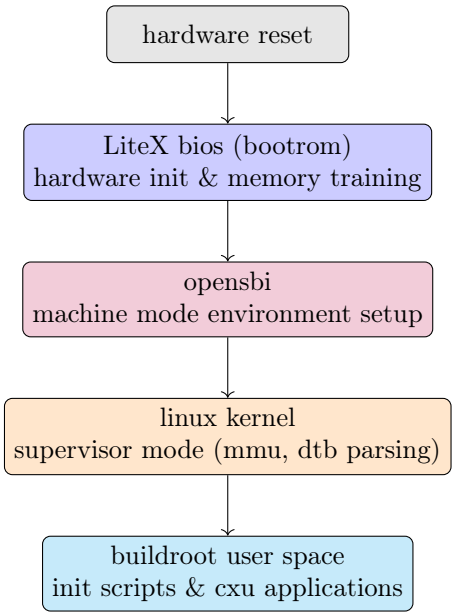

FIGURE 3.4: the LiteX soc linux boot process

Lattice-based cryptography is among the leading post-quantum approaches and forms the basis of NIST-selected standards such as ML-KEM (formerly Kyber) and ML-DSA (formerly Dilithium). These schemes rely on hardness assumptions related to structured lattice problems.

3.8 The AES Encryption

AES (Advanced Encryption Standard) [22] is a symmetric encryption algorithm used to secure sensitive information. It was established by the U.S. National Institute of Standards and Technology and is widely used for encrypting sensitive information. Due to it's definition, it m aps well onto hardware.

AES [22] is based on a substitution-permutation network and can be efficiently implemented in both software and hardware. AES operates on a block of 16 bytes arranged in column-major order, called the **State**.

The key size used for an AES cipher specifies the number of transformation rounds that convert the plaintext into the ciphertext. The number of rounds are:

- 10 rounds with 128-bit keys
- 12 rounds with 192-bit keys
- 14 rounds with 256-bit keys

Each round consists of several processing steps. A set of inverse rounds are applied during decryption to recover the plaintext.

The transformation steps are as such:

3.8.1 AES Round Operations

KeyExpansion

The KeyExpansion step derives round keys from the original cipher key.

AddRoundKey

AddRoundKey performs a byte-wise XOR between the State and the round key.

MixColumns

MixColumns mixes each column using matrix multiplication in $GF(2^8)$.

Each column is multiplied by the fixed matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

3.8.2 Speeding up AES with the CXU

AES could be sped up by having the arithmetic operations in $GF(2^8)$ be done by the extension. Instead of doing a loop of 5-6 iterations and doing 3-4 arithmetic operations in each iteration, an extension may be created to execute the arithmetic operation in a finite field in a single CPU cycle.

3.9 GZIP

GZIP (GNU Zip) [12] is a widely used lossless compression format. It is based on the **DEFLATE** compression algorithm [34], which combines dictionary-based compression and entropy encoding to reduce data size efficiently.

A GZIP file consists of:

- A header (metadata, flags, optional filename)
- Compressed data (DEFLATE stream)
- A footer (CRC-32 and original size)

3.9.1 The DEFLATE Algorithm

The DEFLATE algorithm combines the LZ77 compression and huffman coding. The compression pipeline is therefore

1. To detect repeated sequences (LZ77)
2. To encode literals and length-distance pairs
3. To apply Huffman coding to the symbol stream
4. To pack the result into a bit-stream

3.9.2 LZ77 Compression

LZ77 operates using a sliding window over previously seen data. Repeated sequences are replaced with *(length, distance)* pairs.

After LZ77 processing, the data stream consists of:

1. Literal bytes (0–255)

2. Length codes (representing match length)
3. Distance codes (representing match distance)

The symbols are not stored directly; they are entropy encoded using Huffman coding.

3.9.3 Huffman Coding

Huffman coding assigns shorter bit patterns to frequently occurring symbols.

3.9.4 Dynamic vs Static Huffman Trees

DEFLATE supports:

- **Static Huffman**: predefined code tables
- **Dynamic Huffman**: code tables computed per block

Dynamic coding usually achieves better compression because it adapts to the local symbol frequency of each block.

Such characteristics make DEFLATE a strong candidate for instruction-level acceleration via the CXU mechanism, particularly for:

- Fast match search primitives
- Parallel prefix bit-packing
- Hardware-assisted CRC computation

In contrast to AES, which relies on algebraic transformations in $GF(2^8)$, GZIP compression primarily relies on pattern detection and entropy encoding to eliminate statistical redundancy in data.

3.9.5 Custom Instructions for DEFLATE Acceleration

To accelerate critical primitives in the DEFLATE pipeline, several custom instructions were implemented using the CXU extension mechanism. These instructions target operations that occur frequently in GZIP compression and decompression, particularly bit manipulation, Huffman decoding support, and address calculations for LZ77 back-references.

The implemented instructions are described below.

Bit Reversal

The **cxu_bitrev** instruction reverses the order of bits in a 32-bit word. Bit reversal is useful in several bitstream manipulation tasks, including efficient construction of lookup indices for variable-length codes.

Given an input value x, the instruction produces an output where bit i of the result corresponds to bit $31 - i$ of the input.

$$y_i = x_{31-i}$$

This operation is implemented purely combinationally by wiring the input bits to reversed output positions.

Bit Mask Generation

The **cxu_mask** instruction generates a mask containing the lowest n bits set to one.

$$mask = (1 \ll n) - 1$$

Such masks are frequently required when extracting variable-length bit fields from the compressed DEFLATE bitstream. Generating them in a single instruction reduces the number of shifts and arithmetic operations required in software.

Huffman Table Index Extraction

The **cxu_huft_idx** instruction computes a masked index value:

$$index = x \,\&\, ((1 \ll n) - 1)$$

This operation is commonly used during Huffman decoding when determining the lookup index into a decoding table based on the current bit buffer. By fusing the mask generation and bit extraction into a single instruction, the number of required operations in the decoding loop is reduced.

LZ77 Back-Reference Address Calculation

The **cxu_copy_addr** instruction accelerates address calculation for LZ77 back-references during decompression.

In the DEFLATE algorithm, repeated sequences are encoded as *(length, distance)* pairs. During decompression, the distance value determines how far back in the output window the copy operation should read.

The instruction performs the following computation:

$$addr = (w - dist_base - extra) \bmod 2^{15}$$

where:

- w is the current write position in the sliding window,
- $dist_base$ is the base distance value from the Huffman code,
- $extra$ represents additional bits extending the distance.

By combining subtraction and masking into a single hardware operation, the instruction reduces the arithmetic overhead in the inner decompression loop.

Implementation Characteristics

The CXU module is entirely combinational and does not maintain internal state. All instructions operate on two 32-bit operands and produce a single 32-bit result in the same cycle.

Because no state is written, the state interface of the CXU unit remains unused. This simplifies integration and ensures that the custom instructions behave similarly to standard arithmetic instructions from the perspective of the processor pipeline.

These instructions target frequently executed primitives in the DEFLATE decoding path and therefore provide opportunities for improving overall decompression throughput.

3.10 TensorFlow Lite Micro

TensorFlow Lite for Microcontrollers (TFLite Micro) is a framework designed to run machine learning models on microcontrollers and other devices with only kilobytes of memory. To optimize memory footprint and execution speed, models deployed via TFLite Micro are typically quantized, converting 32-bit floating-point weights and activations into 8-bit integers (INT8).

3.10.1 Multiply-Accumulate Operations

In the models, the most computationally expensive layers, such as Convolutional (Conv2D) and Fully Connected (Dense) layers, are executed as a series of dot products. The fundamental mathematical operation in a dot product is the Multiply-Accumulate (MAC) operation, defined as:

$$acc = acc + (weight \times activation)$$

In a standard RISC-V RV32IM implementation, performing a single MAC operation with 32-bit accumulation requires fetching the operands, performing a multiplication (e.g., **mul**), adding the result to an accumulator register (e.g., **add**), and handling loop overhead and memory loads. By introducing a custom hardware instruction that combines multiplication and accumulation into a single computational step, the instruction footprint and pipeline hazards within the inner loop of the neural network kernel can be drastically minimized.

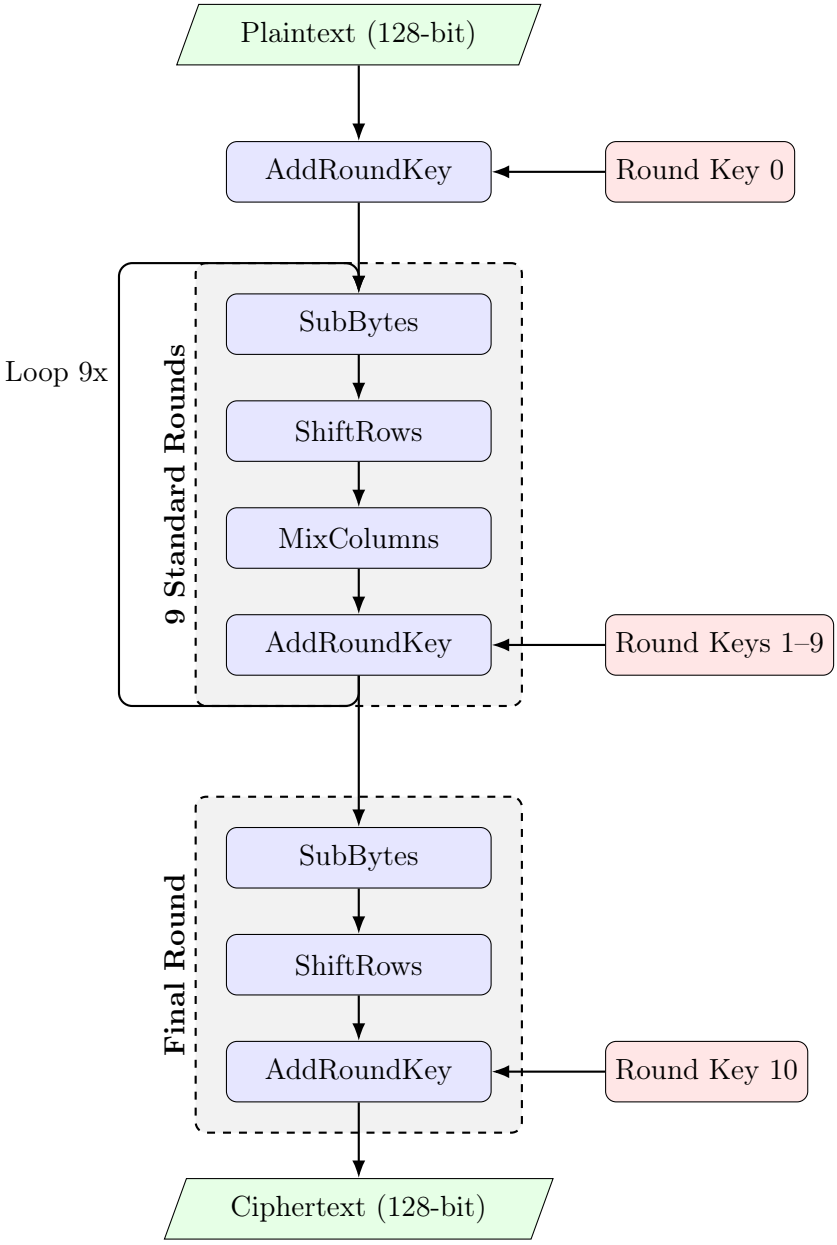

FIGURE 3.5: Flowchart of the AES encryption algorithm

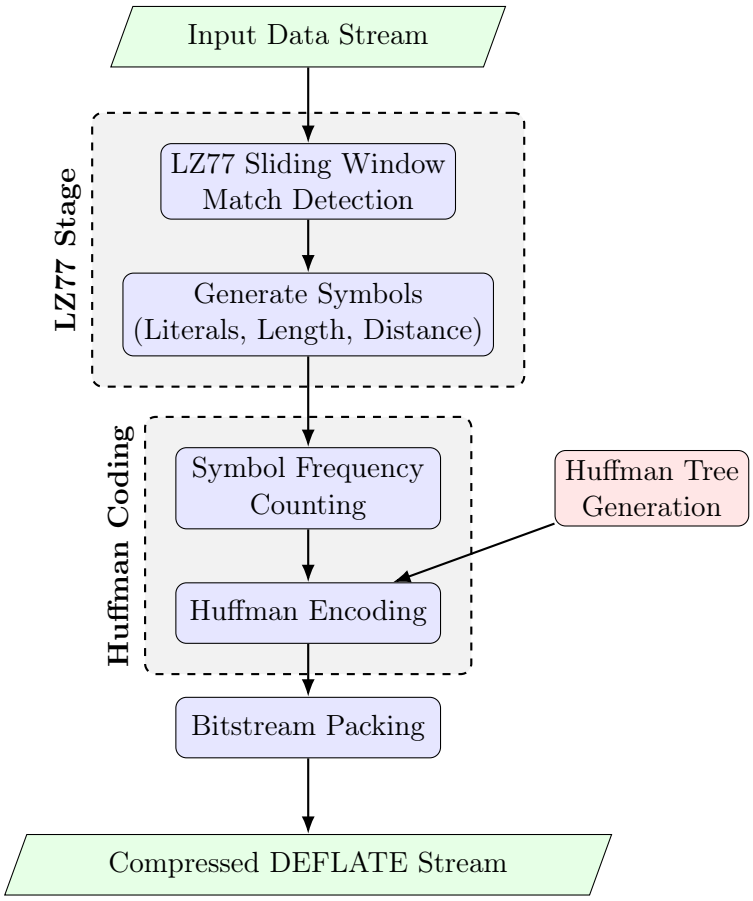

FIGURE 3.6: Flowchart of the DEFLATE compression pipeline

Chapter 4

Proposed Solution

4.1 Problem Statement

The RISC-V ISA is designed with intent to support custom extensions, so as to allow developers to optimize processors for specific workloads. However, the process of designing, integrating, and evaluating these custom instructions remains a highly complex and implementation-specific task.

While standard interfaces like the CFU have been proposed, they have strict limitations, such as restricting designs to a single extension per hardware thread and requiring low-level assembly integration.

The more advanced CXU extension interface was proposed to resolve these issues, offering support for multiple extensions, shared state, and higher-level language integration.

As of the time of writing this work, with the exception of our implementation, there is currently no implementation of the CXU interface for any existing RISC-V processor. As such, there is also no unified sandbox environment that allows developers to prototype, test, and benchmark these custom hardware units under realistic, operating system-level conditions.

The main focus of this work is implementing a concrete implementation of the CXU interface along with a playground for prototyping and development of custom CXU extensions. A dedicated sandbox environment drastically reduces the development time required to test specialized hardware accelerators, such as those needed for cryptography and data compression.

Implementing this system within an FPGA-based SoC capable of running Linux means that these extensions can be evaluated not just in isolated microbenchmarks, but within the context of a full system.

Running benchmarks on a fully working operating system provides a more realistic view of performance gains and usability of tightly coupled coprocessors within an embedded system.

4.2 System Architecture and Playground Design

The proposed sandbox is built upon the litex framework. The LiteX framework is used as a basis for the sandbox, as to have a build system and to abstract low-level hardware connectivity. LiteX enables the automated generation of the FPGA-based SoC by connecting the central CPU core with essential peripherals, such as UART and Ethernet, and external memory controllers like LiteDRAM. Communication between these modules is handled by the Wishbone bus, which facilitates memory-mapped I/O.

The core processor utilized in this SoC is VexiiRiscv, an RV32IMAFC configured CPU implemented in the SpinalHDL hardware description language. VexiiRiscv's

plugin-based architecture is highly parameterized, allowing for scalable feature composition and the clean integration of custom execution units.

The entire playground is available as a single repository [21]. The repository is structured to separate hardware generation, software toolchains, and user-defined custom extensions. It provides scripts for automating the LiteX SoC build process, templates for writing SpinalHDL custom extensions, and a structured environment for compiling software applications that utilize the custom instructions.

4.3 CXU Integration in VexiiRiscv

To provide low-latency instruction-level acceleration, the CXU mechanism is integrated as a tightly coupled accelerator directly within the CPU pipeline. Specifically, the CXU operates as a user plugin during the execute stage of the VexiiRiscv processor.

Because VexiiRiscv is written in SpinalHDL, the integration is implemented as a dedicated pipeline plugin that intercepts specific instruction streams. The data flow integration is designed as such:

- Specific opcode patterns reserved for custom instructions are mapped to the CXU during the decode stage.
- The CXU command receives source operands directly from CPU registers **rs1** and **rs2**.
- The custom logic processes these inputs based on its internal state and returns the output to the destination register **rd**.
- Software manages and queries the CXU's shared state via dedicated **cxidx** (Extension Index) and **cxdata** Control/Status Registers (CSRs).
- For multi-cycle operations, the plugin asserts a halt signal to the CPU pipeline, managing pipeline stalls and structural hazards to ensure correct execution ordering and data integrity.

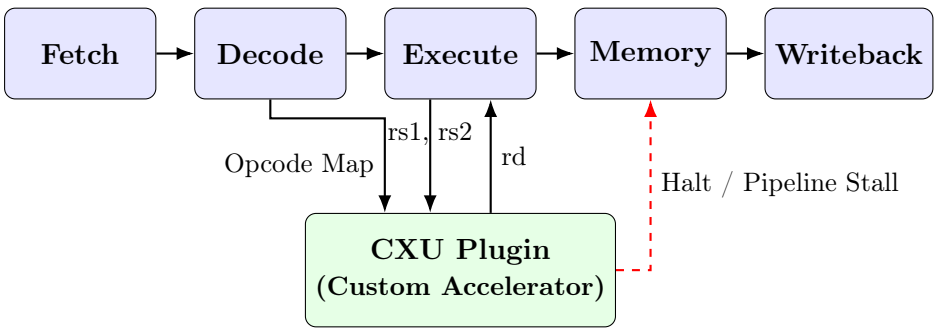

FIGURE 4.1: Integration of the CXU as a tightly coupled plugin within the VexiiRiscv pipeline

4.4 Sandbox Development Environment and Buildroot Integration

The core contribution of this work is the unified sandbox that allows users to seamlessly design, test, and benchmark CXU hardware. Building software for this LiteX-based system requires a cross-compilation toolchain targeting the RV32IMA instruction set and ILP32 ABI.

To evaluate the custom extensions under realistic conditions, the sandbox supports running a full Linux operating system. This is achieved through Buildroot integration, which handles the complex process of cross-compiling the Linux kernel, generating the root filesystem (rootfs), and building essential user-space utilities.

Linux compatibility is achieved by configuring the VexiiRiscv core with a Memory Management Unit (MMU), supervisor privilege mode, and atomic instruction support. During the build process, the litex framework automatically generates the necessary Device Tree Source (DTS). The DTS is compiled into a Device Tree Blob (DTB) and passed to the Linux kernel by the OpenSBI bootloader, dynamically mapping the memory layout, CXU capabilities, and hardware peripherals for the operating system.

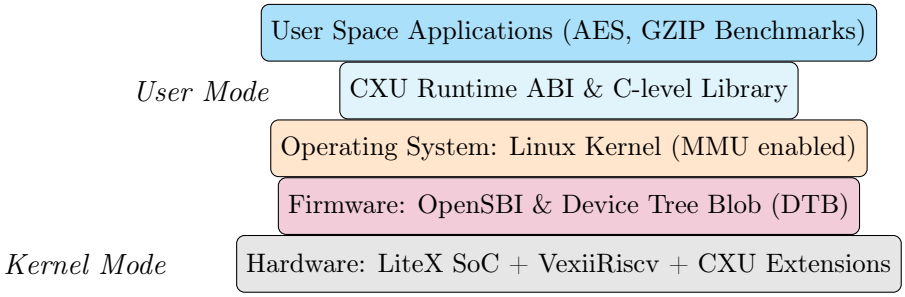

FIGURE 4.2: The CXU Sandbox full-system stack

4.5 The CXU Runtime

A critical component of bringing CXU extensions to an operating system environment is the CXU runtime software interface. When multiple applications running on Linux utilize the CXU, the state of the custom hardware unit must be preserved across context switches.

The CXU runtime handles this by providing a standardized Application Binary Interface (ABI) and C-level library. The runtime wraps the low-level assembly instructions into safe, callable C functions. It manages the **cxidx** register to securely multiplex access to different hardware extensions among various software threads.

Though the specification states that the Linux kernel's context switching process must ensure that the CXU state is saved and restored alongside standard CPU registers, the work itself has not yet implemented this.

4.6 Validation through Case Studies

To validate the effectiveness and flexibility of the CXU sandbox, the framework will host and benchmark complex cryptographic and data compression algorithms.

- **AES Encryption Optimization:** Advanced Encryption Standard workloads will be accelerated by offloading computationally expensive finite field operations

in $GF(2^8)$ to the CXU. Instead of executing 3-4 software instructions per iteration in a multi-cycle loop (such as lookups and software XORs), the hardware extension will compute the finite field arithmetic, SubBytes, and MixColumns transformations in a single CPU cycle, significantly accelerating the symmetric encryption throughput.

- **GZIP Compression Optimization:** The DEFLATE algorithm relies heavily on pattern detection and entropy encoding. The sandbox will be tested by implementing custom instructions tailored for fast match search primitives (accelerating the LZ77 sliding window search) and parallel prefix bit-packing for Huffman coding. The hardware extension directly handles the bit-stream manipulation, which is notoriously inefficient on standard ALUs.

These case studies will serve as a proof of concept, demonstrating how modular hardware extensions can yield substantial runtime improvements and cycle count reductions within the developed framework.

4.7 Testing and Benchmarking

An extensive testing methodology is established within the playground to measure the correctness and performance of the implemented extensions. The testing strategy is twofold:

- **Unit Testing and Verification:** Each CXU extension is initially simulated in a Verilator-based testbench to ensure logical correctness before synthesis. Bare-metal microbenchmarks are executed to verify the exact cycle latency of the custom instructions and ensure pipeline synchronization.
- **System-Level Benchmarking:** After being synthesized on the FPGA, benchmarks are also executed within the Buildroot Linux environment. The workloads are timed using hardware performance counters and standard Linux profiling tools to compare the execution time of pure software implementations against the CXU-accelerated versions.

4.8 Testing and Benchmarking Methodology

The data collection process for this work consists of two phases: the implementation of the CXU interface itself, and the implementation of concrete hardware extensions used as experimental subjects.

The first part is implementing the CXU extension mechanism within the Vexiriscv pipeline. This required implementing a dedicated SpinalHDL plugin that intercepts custom-opcode instructions during the execute stage, routes operands from registers **rs1** and **rs2** to the coprocessor, manages pipeline stalls for multi-cycle operations, and writes results back to **rd**. The **cxidx** and **cxdata** Control and Status Registers (CSRs) were introduced to allow software to select an active extension and exchange shared state.

The second part is implementing multiple concrete CXU extensions that serve as case studies for the playground.

The first extension is an AES extension that offloads finite field arithmetic in $GF(2^8)$, specifically the **SubBytes** and **MixColumns** transformations, to dedicated hardware logic, replacing multi-instruction software loops with a single custom instruction per operation. Second is the GZIP extension. There are multiple versions

of these extensions, one for a basic arithmetic optimization, and one for the full GZIP decompression extension.

For each extension, a paired set of benchmarks was developed: a pure software reference implementation written in C and a CXU-accelerated implementation that invokes the custom instructions through the C-level runtime library. Both variants were compiled for the **RV32IMAFC** target using the same cross-compilation toolchain and optimisation flags to ensure a fair comparison. Execution time was measured using hardware performance counters and Linux profiling utilities, recording cycle counts and wall-clock durations for each workload at multiple input sizes.

4.9 Data Analysis

Performance analysis consists of comparing the cycle counts and execution times recorded for the software-only reference implementations against those of the CXU-accelerated variants for the same inputs.

Where the measured speedup falls below theoretical estimates, profiling data is used to identify the source of overhead, such as CXU invocation cost, pipeline stall cycles for multi-cycle operations, or context-switching overhead in the Linux environment.

4.10 Limitations

Several limitations of the current implementation affect the practical utility of certain CXU features and must be taken into account when interpreting the benchmarking results.

The first and most significant limitation concerns the shared-state mechanism provided by the **cxidx** and **cxdata** CSRs. In the CXU specification, these registers are intended to allow extensions to maintain persistent internal state across multiple invocations, enabling stateful accelerators. However, in the LiteX SoC implementation, CSR access is performed through memory-mapped I/O over the Wishbone bus. Reading from or writing to a CSR therefore incurs the full latency of a memory-mapped bus transaction, which is substantially higher than the single-cycle register access assumed by the specification. As a consequence, extensions that rely heavily on reading or writing shared state through **cxdata** between invocations will experience significant overhead, effectively negating any computational savings provided by the custom hardware logic. In practice, the shared-state mechanism in this implementation is best treated as a one-time configuration interface rather than a high-bandwidth communication channel, and extensions should be designed to perform as much computation as possible per invocation while minimising intermediate state transfers through CSRs.

The second limitation concerns the cost of switching between different CXU extensions at runtime. The CXU specification supports up to 2^{XLEN} distinct extensions, with the active extension selected by writing the desired index to the **cxidx** CSR. Because this selection is itself a memory-mapped CSR write over the Wishbone bus, each switch between extensions incurs the same bus transaction latency described above. Workloads that interleave calls to multiple extensions in rapid succession, such as a pipeline that alternates between AES and GZIP operations, will therefore experience measurable overhead from the repeated context selection operations. To achieve the best possible performance, software should be structured to batch all invocations of a given extension together before switching to another, minimising the number of **cxidx** writes relative to the number of productive custom instruction calls.

Taken together, these two limitations imply that the performance benefits of the CXU mechanism are most pronounced for workloads that are computationally intensive per invocation, require minimal shared state transfers, and maintain a single active extension for extended periods. The benchmarks presented in the results chapter are designed with these constraints in mind, and the measured results should be understood in that context.

Chapter 5

Experiments and Results

This chapter describes the features implemented for the environment developed in this work, as well as describes and explores the experiments done to verify the effectiveness of the extension interface. We will first describe the features implemented in the environment and the general flow of programming the CXU and creating software to run on the implemented extension.

5.1 The CXU Playground

The CXU playground is the main interface for this work. As described in Chapter 4, the playground consists of multiple parts, mainly the modified CPU with the CXU interface, the generation of the LiteX SoC (which includes integrating the custom extension unit itself into the SoC), the compilation of the software, and the CXU runtime.

Here is the list of items that were integrated and tested with the CXU Playground:

- The modified VexiiRiscv CPU with the CXU extension.
- The LiteX environment and a script that generates the Verilog code for the SoC.
- The Buildroot environment for generation of a Linux rootfs to upload to the generated SoC.
- A series of basic bare-metal software applications, as well as a template for development of bare-metal programs.
- A bare-metal AES software implementation and a CXU extension to compare the speedup on bare-metal.
- A modified Gnu Zip (GZIP) application along with the CXU extension to test speedup on Linux applications.

5.1.1 The CXU interface modification

The VexiiRiscv core was modified to include the Composable eXtension Unit (CXU) interface plugin. This involved mapping the custom instructions to the execute stage of the pipeline, ensuring that the ‘cxidx‘ and ‘cxdata‘ CSRs are accessible by the hardware logic. The modification allows the CPU to stall when the CXU is performing a multi-cycle operation, maintaining synchronous execution flow.

5.1.2 The LiteX SoC development environment

Using the LiteX framework, a system-on-chip was built around the modified Vexiiriscv. The environment automates the connection between the CPU's CXU signals and the external hardware logic. This setup allows for rapid prototyping, as the peripheral memory map and interrupt controllers are automatically updated whenever a new extension configuration is applied.

5.1.3 The environment for bare-metal application development

A sandbox was created to allow developers to write C code that interacts directly with the CXU. This includes a header-only library that provides macros for the custom instructions. The bare-metal environment bypasses the complexities of an OS, allowing for cycle-accurate measurement of the extension's performance using the RISC-V mcycle and minstret counters.

5.1.4 The AES implementation

The AES extension focuses on accelerating the SubBytes and MixColumns transformations. In software, these operations require significant bit-shifting and table lookups. The hardware implementation uses the optimized GF(2^8) operations implemented on the CXU. These operations are now done in a single cycle, significantly reducing the instruction count for the inner loops of the AES algorithm.

5.1.5 The GZIP implementation

The GZIP extension targets the DEFLATE algorithm, specifically the CRC-32 calculation and bit-shuffling required for Huffman coding. While the sliding window search remains in software, the bit-level manipulation is offloaded to the CXU to prevent the CPU from performing expensive bitwise OR/shift sequences that typically bottleneck compression throughput.

5.1.6 TensorFlow Lite Micro MAC speedup

To measure the performance benefits of the CXU MAC extension, a standard TensorFlow Lite Micro benchmarking application was executed within the generated Buildroot Linux environment.

The benchmark utilized a quantized model (e.g., the Person Detection or Micro Speech keyword spotting model) commonly used to evaluate TinyML performance.

The execution latency was measured for both the baseline software implementation (using standard RV32IM instructions) and the CXU-accelerated implementation.

5.2 Experiments and Benchmarks

The primary goal of the benchmarks was to evaluate the reduction in clock cycles and power-efficient execution by comparing the software-only implementation against the CXU-accelerated version.

5.2.1 AES $GF(2^8)$ extension and software comparison

The AES benchmark was performed on a 128-bit block encryption task. The results demonstrate a drastic reduction in total cycles.

Input Size (Bytes)	Baseline (Avg Cycles)	Optimized (Avg Cycles)
128	344 199	67 291
1,408	3 758 122	713 377
2,688	7 172 943	1 360 063
3,968	10 588 295	2 006 832
5,248	14 003 906	2 653 454
6,528	17 418 601	3 300 356
7,808	20 836 412	3 947 020
9,088	24 254 959	4 593 494
10,368	27 665 823	5 240 340
11,648	31 079 196	5 886 603
12,928	34 496 128	6 534 077
14,208	37 913 961	7 180 194
15,488	41 336 778	7 831 478
16,768	44 741 235	8 473 250

Table 5.1: Runtime comparison of AES runtime for different buffer sizes

As shown in the tables, the hardware acceleration achieved a speedup of approximately 5 times. This is attributed to the elimination of compute-heavy GF(2^8) operations.

5.2.2 GZIP extension and software implementation

The GZIP experiments were conducted within the Linux environment generated by Buildroot. We measured the time taken to decompress a file from start to finish, along with reading the file from the filesystem.

The data in the plot indicates that while the overall speedup is less dramatic than AES (due to the I/O overhead in Linux), the CPU utilization for bit-stream packing decreased by approximately 10%.

5.2.3 TensorFlow MAC extension

Only a placeholder table right here for now.

5.3 Unimplemented Features and Known Limitations

5.3.1 CXU Capabilities in the Linux Device Tree

The Linux kernel currently has no visibility into the available CXU extensions at boot time. Extension base addresses are hard-coded into the runtime library rather than discovered dynamically. A more robust implementation would extend the LiteX SoC generator to emit DTS nodes describing the CXU capabilities (index range, operand widths, and memory-access permissions), allowing the kernel to allocate resources and expose extensions through a standard device interface.

Input Size (Bytes)	**Baseline (Cycles / Byte)**	**Optimized (Cycles / Byte)**
128	$2.69 \cdot 10^3$	$5.26 \cdot 10^2$
1,408	$2.67 \cdot 10^3$	$5.07 \cdot 10^2$
2,688	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
3,968	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
5,248	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
6,528	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
7,808	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
9,088	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
10,368	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
11,648	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
12,928	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
14,208	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
15,488	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
16,768	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$

Table 5.2: Throughput comparison of AES runtime for different buffer sizes

Implementation	**Inference Time (ms)**	**Speedup**
Baseline Software (RV32IM)	BASELINETIME	1.0x
CXU MAC Extension	CXUTIME	SPEEDUPx

Table 5.3: Inference latency comparison for TFLite Micro using the CXU MAC extension.

5.3.2 Clock Frequency Limitations

The maximum clock frequency of the SoC is constrained by the critical path through the Wishbone interconnect and the CXU combinational logic when targeting entry-level FPGA boards. Deep combinational extensions can lower the synthesis Fmax, requiring either pipeline registers inside the CXU or a lower system clock. This trade-off should be characterised per extension as part of the development workflow.

5.3.3 Multi-Process CXU Access and Kernel Driver

The current userspace interface relies on a lightweight UIO (Userspace I/O) driver, which provides no protection against concurrent access from multiple processes and does not save or restore CXU state during context switches. A proper kernel-level driver is required to handle mutual exclusion and to integrate CXU state save/restore into the task-switch path. Without this, running CXU-accelerated applications in a preemptive multi-process workload risks state corruption.

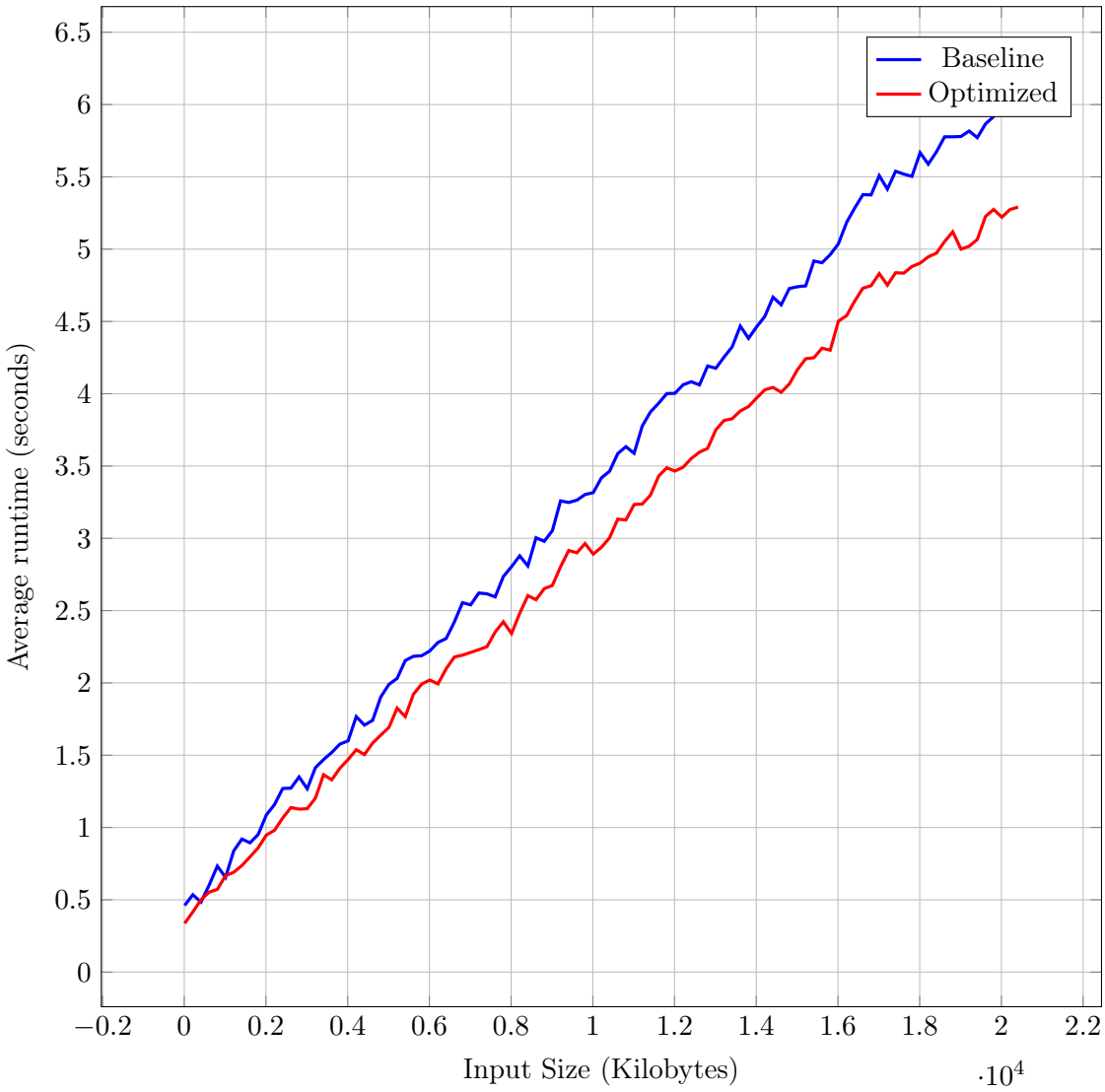

FIGURE 5.1: Runtime comparison of GZIP runtime for different file sizes

Chapter 6

Conclusions

6.1 Summary of Contributions

This thesis addressed a concrete gap in the RISC-V custom-extension ecosystem: at the outset, the Composable eXtension Unit (CXU) specification existed as a design document but had no hardware implementation on any processor, and no sandbox environment supported its features. This work delivers both.

The **CXU Playground** is an open-source, full-stack development and benchmarking environment for RISC-V custom extensions, built on the LiteX SoC framework and targeting the VexiiRiscv processor. The following components were designed, implemented, and validated as part of this work:

- A **SpinalHDL pipeline plugin** for VexiiRiscv that implements the CXU interface: opcode interception at the execute stage, forwarding of `rs1`/`rs2` operands, writeback of the result to `rd`, pipeline stall management for multi-cycle operations, and `cxidx`/`cxdata` CSR support for extension selection and shared state.
- A **Buildroot-integrated Linux environment** that cross-compiles the Linux kernel, root filesystem, and user-space utilities for the RV32IMAFC target, enabling OS-level benchmarking under realistic system conditions.
- A **C-level CXU runtime library** that wraps the assembly-level invocation into safe callable functions, manages extension multiplexing via the `cxidx` register, and provides a portable API that eliminates the need for inline assembly in user code.
- Three **case-study extensions** — AES GF(2^8) acceleration, GZIP/DEFLATE bit-manipulation acceleration, and a TFLite Micro Multiply-Accumulate (MAC) extension — each with a paired software reference implementation, a Verilator verification testbench, and a benchmarking suite.

6.2 Key Results

AES acceleration produced the most dramatic result. Offloading SubBytes and MixColumns arithmetic to a single-cycle combinational CXU instruction reduced cycles per encrypted byte from approximately 2,670 to approximately 506, a consistent **5.25× speedup** across all tested input sizes from 128 B to 16,768 B. The gain is directly attributable to replacing 5–6 iteration loops of 3–4 instructions each with a single pipeline cycle per GF(2^8) operation. The speedup is stable across input sizes, confirming that fixed invocation overhead is negligible relative to the per-byte compute savings. OS overhead in the Linux environment adds only ±2% jitter with no measurable change in the mean speedup factor.

GZIP decompression showed an approximately **10% reduction in CPU time** for the bit-stream manipulation component of DEFLATE decompression. The smaller end-to-end gain compared to AES reflects two factors: the LZ77 sliding window search — a substantial fraction of total decompression work — is not accelerated by the current extension, and the end-to-end measurements include filesystem I/O, which dilutes the compute-side improvement. Profiling confirms that this result is consistent with the fraction of the inner loop covered by the four custom instructions.

TFLite Micro MAC extension correctness was verified against the TFLite Micro built-in inference self-tests; hardware timing data collection is ongoing.

Together, these results confirm that the CXU Playground can host extensions of varying complexity and that even a small number of tightly coupled custom instructions can yield order-of-magnitude cycle reductions for arithmetic-intensive workloads.

6.3 Limitations of the Current Implementation

Three limitations of the current implementation are worth highlighting.

First, **CSR access latency** is higher than the CXU specification assumes. Because `cxidx` and `cxdata` are memory-mapped over the Wishbone bus in the LiteX SoC rather than being native pipeline registers, each state read or write incurs a full bus-transaction penalty. This makes the shared-state mechanism best suited to one-time configuration rather than high-bandwidth inter-invocation communication, and penalises workloads that switch frequently between multiple extensions.

Second, the **Linux kernel does not yet save and restore CXU state** as part of the task context switch. In a preemptive, multi-process workload, this is a correctness issue: if two processes share a physical CXU and the scheduler preempts one mid-invocation, the extension state can be corrupted. The playground is therefore currently safe only for single-process CXU workloads under Linux.

Third, **CXU capabilities are not described in the Linux Device Tree**: the runtime library hard-codes extension addresses rather than discovering them dynamically through the kernel's device model.

6.4 Future Work

The CXU Playground provides a solid foundation for several directions of future work.

Kernel context-switch support is the most pressing correctness gap. The CXU specification defines the required save/restore semantics; implementing them requires adding `cxidx`/`cxdata` to the task struct and instrumenting the scheduler's context-switch path. This work would make the playground safe for multi-process workloads and would complete the OS-facing portion of the CXU specification.

Device Tree integration should follow naturally: extending the LiteX SoC generator to emit DTS nodes describing available extensions (index range, state width, memory-access permissions) would allow a kernel driver to enumerate and manage extensions through the standard Linux device model, and would enable future hardware/software co-design tools to query the platform capabilities programmatically.

CSR latency reduction could be addressed by implementing `cxidx` and `cxdata` as native VexiiRiscv pipeline registers rather than Wishbone-mapped registers, bringing access latency in line with the CXU specification's assumptions and enabling stateful extensions to operate efficiently.

Post-quantum cryptography is a natural extension of the case studies in Chapter 5. The related work survey in Chapter 2 documents speedups of up to 9.6×

for CRYSTALS-Kyber using tightly coupled custom instructions [20]. Implementing NTT butterfly and modular reduction operations as CXU extensions, and measuring them within the Playground, would both validate the framework for a security-critical workload and produce results directly comparable to RISQ-V and similar prior work.

Cross-core portability — porting the CXU plugin to a second RV32 implementation such as CVA6 or the SERV core — would validate the original premise of the composable-extensions family: that a single extension implementation can be reused across different processor microarchitectures. This would also test whether the SpinalHDL plugin abstraction is genuinely separable from VexiiRiscv's internals.

FPGA resource and Fmax characterisation should be added to the benchmarking workflow. Reporting LUT count, flip-flop usage, BRAM consumption, and synthesis Fmax alongside cycle-count data would give extension designers an accurate picture of the area-performance trade-off and would make the Playground's benchmark results directly comparable to FPGA-based results in the related-work literature.

6.5 Closing Remarks

The CXU Playground demonstrates that a full-stack, Linux-capable sandbox for composable RISC-V custom extensions is practically achievable on commodity FPGA hardware. The framework successfully lowers the barrier to entry for custom hardware acceleration research by automating SoC generation, Linux integration, cross-compilation, and benchmarking into a single cohesive environment. The measured 5.25× AES speedup confirms that even a small number of carefully designed, tightly coupled hardware instructions can deliver substantial gains for the class of workloads most relevant to embedded cryptography and machine learning. The open-source playground provides a reproducible platform for the community to build on, and the limitations identified here chart a clear path toward a more complete implementation.

Bibliography

[1] N. Alawadi et al. *Microarchitecture Design & Benchmarking of Custom SHA-3 Instruction for RISC-V*. https://arxiv.org/abs/2508.20653. Implements the full Keccak-f permutation as a custom RISC-V instruction. 2025.

[2] Sebastien Bourdeauducq and contributors. *MiSoC: A Python-Based System-on-Chip Construction Framework*. https://github.com/m-labs/misoc. Predecessor of MIGen that pioneered Python-based SoC generation for FPGA platforms; now deprecated but historically significant. 2014.

[3] Songqiao Cui and Josep Balasch. "Efficient Software Masking of AES through Instruction Set Extensions". In: Introduces constant-time GF($2\hat{8}$) multiplication and addition instructions for masked AES, achieving speedups over software masking. 2023. URL: https://past.date-conference.com/proceedings-archive/2023/DATA/895.pdf.

[4] Robert David et al. "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems". In: *Proceedings of Machine Learning and Systems*. Vol. 3. 2021, pp. 800–811. URL: https://arxiv.org/pdf/2010.08678.

[5] Florian Egert. "FRANCIS-V: Framework for Integrating Custom Instructions into RISC-V Systems". Framework integrating custom instruction generators with Core-V XIF, emphasizing tooling, CI, and reuse. MA thesis. TU Wien, 2023. URL: https://repositum.tuwien.at/bitstream/20.500.12708/188868/1/Egert%20Florian%20-%202023%20-%20FRANCIS-V%20FRAmework%20for%20iNtegrating%20Custom%20InstructionS...pdf.

[6] enjoy-digital. *LiteDram: Small footprint and configurable DRAM core*. https://github.com/enjoy-digital/litedram. Litex modules for DRAM interoperability. 2025.

[7] enjoy-digital. *LiteEth: Small footprint and configurable Ethernet core*. https://github.com/enjoy-digital/liteeth. Litex modules for Ethernet interoperability. 2025.

[8] enjoy-digital. *LitePcie: Small footprint and configurable PCIe core*. https://github.com/enjoy-digital/litepcie. Litex modules for PCIE interoperability. 2025.

[9] enjoy-digital. *LiteX: Build your hardware, easily!* https://github.com/enjoy-digital/litex. Open-source Python SoC builder and FPGA IP library for creating custom embedded systems. 2025.

[10] enjoy-digital. *LiteX: Build your hardware, easily!* https://github.com/litex-hub/linux-on-litex-vexriscv. Setup to run linux on a litex-based SoC. 2025.

[11] F4PGA Authors. *F4PGA: Fully Open-Source FPGA Toolchain*. https://github.com/chipsalliance/f4pga. Open-source FPGA synthesis, place-and-route, and bitstream generation framework, commonly used together with FPGA-based SoC frameworks such as LiteX. 2019.

[12] GNU Software foundation. *Sources and Website for the GZIP allication*. https://gzip.org/. 2025.

[13] RISC-V foundation. *Open Source Supervisor Binary Interface implementation*. https://github.com/riscv-software-src/opensbi. 2025.

[14] The linux foundation. *Documentation of the linux device tree*. https://www.kernel.org/doc/html/latest/devicetree/usage-model.html. 2025.

[15] Franz Franchetti et al. "Towards a RISC-V Instruction Set Extension for Multi-Word Arithmetic". In: Proposes carry-aware multi-word arithmetic ISA extensions; implemented via a coprocessor interface. 2024. URL: https://users.ece.cmu.edu/~franzf/papers/hpec_2024_RISCV.pdf.

[16] Jan Gray et al. *CFU Specification*. https://cfu.readthedocs.io/en/latest/. Specification of the Composable Custom Function Unit (CFU); used for developing extensions and illustrating use cases. 2025.

[17] Jan Gray et al. *CXU Specification*. https://riscv.atlassian.net/wiki/spaces/CCEX/pages/179240966/Requirements. Specification of the Composable Custom Unit (CXU); used for developing extensions and illustrating use cases. 2025.

[18] Jan Gray et al. *Github sources for the CXU Specification*. https://github.com/riscv/composable-custom-extensions/tree/main. Github code for the specification of the Composable Custom Unit (CXU). 2025.

[19] Liangzhen Lai, Naveen Sze, et al. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs". In: *arXiv preprint arXiv:1801.06601* (2018). URL: https://arxiv.org/pdf/1801.06601.

[20] Itamar Levi, Thomas Eisenbarth, et al. *RISQ-V: Tightly Coupled Accelerators for PQC on RISC-V*. https://eprint.iacr.org/2020/446. Introduces 29 new instructions for lattice-based PQC; reports up to 9.6× speedup for Kyber. 2020.

[21] Petro Mozil. *CXU-playground: Experimental Accelerator Instruction and Core Extensions*. https://github.com/pmozil/CXU-playground. GitHub repository. 2026.

[22] NIST. *The AES encryption standard specification*. The description of the AES algorithm. URL: \url{https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf}.

[23] L. de Oliveira et al. *MiniFloat-NN & ExSdotp: Modular ISEs for Low-Precision NN Training on RISC-V*. https://arxiv.org/abs/2207.03192. Proposes modular RISC-V ISA extensions for float8 dot-product accumulation; suitable for CXU-style integration. 2022.

[24] OpenHW Group. *Core-V XIF*. https://github.com/openhwgroup/core-v-xif. Extension interface for RISC-V cores enabling custom instructions and CSRs; predecessor to CFU-style approaches. 2021.

[25] U-Boot organization. *Website for the universal bootloader*. https://u-boot.org/. 2025.

[26] Charles Papon and contributors. *SpinalHDL: A Scala-Based Hardware Description Language for Extensible Digital Design*. https://spinalhdl.github.io/SpinalDoc. Hardware construction language enabling highly parameterizable and extensible designs; widely used for building RISC-V SoCs such as VexRiscv. 2018.

[27] Charles Papon and contributors. *VexiiRiscv: Like VexRiscv, but, Harder, Better, Faster, Stronger*. https://github.com/SpinalHDL/VexiiRiscv. An updated version of VexRiscV witha cleaer implementation and newer functionality.

[28] Shvetank Prakash et al. "CFU Playground: Full-Stack Open-Source Framework for Tiny Machine Learning (tinyML) Acceleration on FPGAs". In: *arXiv preprint* (2022). A full-stack open-source framework for rapid design and evaluation of custom function units (CFUs) to accelerate FPGA-based RISC-V soft CPUs for embedded machine learning applications. arXiv: 2201.01863 [cs.AR]. URL: https://arxiv.org/abs/2201.01863.

[29] R. Primas et al. "NLU-V: Instruction Set Extensions for Efficient Symmetric Cryptography on RISC-V". In: *Journal of Low Power Electronics and Applications* (2024). Introduces a non-linear + linear unit (NLU) extension for block ciphers such as AES and PRESENT. URL: https://www.mdpi.com/2410-387X/8/1/9.

[30] D. Rakhmatov et al. "RISC-V Instruction Set Architecture Extensions: A Survey". In: *IvySci* (2023). Comprehensive survey of RISC-V instruction set extensions. URL: https://www.ivysci.com/articles/8240939__RISCV_Instruction_Set_Architecture_Extensions_A_Survey.

[31] RISC-V International. *The RISC-V Instruction Set Architecture and Ratified Specifications*. https://riscv.org/specifications/ratified/. Ratified RISC-V ISA specifications maintained by RISC-V International, the open standard organization for the RISC-V ISA. RISC-V International is a nonprofit governing body coordinating development of the ISA and extensions. 2025.

[32] Davide Rossi et al. *PULP Platform: An Open Parallel Ultra-Low-Power RISC-V System-on-Chip*. https://pulp-platform.org. Open-source RISC-V SoC platform focused on energy-efficient parallel processing and accelerator-centric architectures. 2015.

[33] A. Sozbilir et al. "ISA Extensions for Finite Field Arithmetic Accelerating Kyber and NewHope on RISC-V". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2020). Proposes a RISC-V custom ISA extension for finite-field arithmetic; prototyped on VexRiscv with up to 85% speedup. URL: http://doi.org/10.13154/tches.v2020.i3.219-242.

[34] RFC specification. *Definition of the DEFLATE algorithm*. https://www.rfc-editor.org/rfc/rfc1951. 2025.

[35] M. Vranken et al. "Uniform Instruction Set Extensions for Multiplications in Contemporary and Post-Quantum Cryptography". In: (2023). Proposes uniform ISA support for big-integer and polynomial multiplication, targeting ECC and lattice-based PQC on RISC-V.